

Topic 5: Sentinels

Contents

- [layout.csl](#)
- [pe_program.csl](#)
- [sentinel.csl](#)
- [run.py](#)
- [commands.sh](#)

In previous programs, we used so-called routable colors, which are associated with a route to direct the flow of wavelets. On WSE-2, task IDs which can receive data wavelets are in the range 0 through 23, corresponding to the IDs of the colors. On WSE-3, task IDs which can receive data wavelets are in the range 0 through 7, corresponding to input queues which are bound to a routable color. We have also used local tasks, which on WSE-2 can be associated with any task ID from 0 to 30, and on WSE-3 can be associated with any task ID from 8 to 30.

This example demonstrates the use of a non-routable control task ID to signal the end of an input tensor. We call this use for a control task ID a *sentinel*.

In this example, the host sends to a receiving PE ([sentinel.csl](#)) the number of wavelets that the receiving PE should expect to receive, followed by the stream of data. The receiving PE then sends the data to its neighbor ([pe_program.csl](#)), followed by a *control wavelet* which specifies the control task ID that the neighbor will activate.

Since sentinel control task IDs are not routable colors, the programmer does not specify a route, but does need to bind the control task ID to a control task, which will be activated upon receipt of the sentinel wavelet. Here, the sentinel activates the [send_result](#) task, which relays the result of the sum reduction back to the host.

layout.csl

```
// Color map
//
// ID var      ID var  ID var      ID var
// 0 main_color   9      18          27 reserved (memcpy)
// 1             10     19          28 reserved (memcpy)
// 2 MEMCPYH2D_1 11     20          29 reserved
// 3 MEMCPYH2D_2 12     21 reserved (memcpy) 30 reserved (memcpy)
// 4 MEMCPYD2H_1 13     22 reserved (memcpy) 31 reserved
// 5             14     23 reserved (memcpy) 32
// 6             15     24          33
// 7             16     25          34
```

// 8 17 26 35

// See task maps in sentinel.csl and pe_program.csl

```
//          +-----+          +-----+
// MEMCPYH2D_1 -> | sentinel.csl | -> main_color -> | pe_program.csl | -> MEMCPYD2H_1
// MEMCPYH2D_2 -> |           |          +-----+
//          +-----+
```

// IDs for memcpy streaming colors

param MEMCPYH2D_DATA_1_ID: i16;

param MEMCPYH2D_DATA_2_ID: i16;

param MEMCPYD2H_DATA_1_ID: i16;

// number of PEs in a column

param size: i16;

// Sentinel to tell PE that it is time to send the result to the host

const end_computation: u16 = 43;

// Colors

const main_color: color = @get_color(0);

```
const memcpy = @import_module("<memcpy/get_params>", .{
    .width = 2,
    .height = size,
    .MEMCPYH2D_1 = MEMCPYH2D_DATA_1_ID,
    .MEMCPYH2D_2 = MEMCPYH2D_DATA_2_ID,
    .MEMCPYD2H_1 = MEMCPYD2H_DATA_1_ID
});
```

layout {

@set_rectangle(2, size);

```
for (@range(u16, size)) |idx| {
    @set_tile_code(0, idx, "sentinel.csl", .{
        .memcpy_params = memcpy.get_params(0),
        .main_color = main_color,
        .sentinel = end_computation,
    });
```

@set_color_config(0, idx, main_color, .{ .routes = .{ .rx = .{ RAMP }, .tx = .{ EAST }}});

```
@set_tile_code(1, idx, "pe_program.csl", .{
    .memcpy_params = memcpy.get_params(1),
    .main_color = main_color,
    .sentinel = end_computation,
});
```

@set_color_config(1, idx, main_color, .{ .routes = .{ .rx = .{ WEST }, .tx = .{ RAMP }}});

}

pe_program.csl

```
// WSE-2 task ID map
// On WSE-2, data tasks are bound to colors (IDs 0 through 24)
//
// ID var          ID var  ID var          ID var
// 0 main_task_id   9       18              27 reserved (memcpy)
// 1                10      19              28 reserved (memcpy)
// 2                11      20              29 reserved
// 3                12      21 reserved (memcpy) 30 reserved (memcpy)
// 4                13      22 reserved (memcpy) 31 reserved
// 5                14      23 reserved (memcpy) 32
// 6                15      24              33
// 7                16      25              34
// 8                17      26              35
// ...
// 43 sentinel_task_id

// WSE-3 task ID map
// On WSE-3, data tasks are bound to input queues (IDs 0 through 7)
//
// ID var          ID var  ID var          ID var
// 0 reserved (memcpy) 9       18              27 reserved (memcpy)
// 1 reserved (memcpy) 10      19              28 reserved (memcpy)
// 2 main_task_id     11      20              29 reserved
// 3                  12      21 reserved (memcpy) 30 reserved (memcpy)
// 4                  13      22 reserved (memcpy) 31 reserved
// 5                  14      23 reserved (memcpy) 32
// 6                  15      24              33
// 7                  16      25              34
// 8                  17      26              35
// ...
// 43 sentinel_task_id

param memcpy_params;

const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

// Sentinel to signal end of data
param sentinel: u16;

// Colors
param main_color: color;

// Queue IDs
const main_iq: input_queue = @get_input_queue(2);
const d2h_1_oq: output_queue = @get_output_queue(3);

// Task IDs
// On WSE-2, data task IDs are created from colors; on WSE-3, from input queues
const main_task_id: data_task_id =
    if      (@is_arch("wse2")) @get_data_task_id(main_color)
    else if (@is_arch("wse3")) @get_data_task_id(main_iq);
```

```

const send_result_task_id: control_task_id = @get_control_task_id(sentinel);

// Accumulate all received values along main_color in result[0]
var result = @zeros([1]f32);
const result_dsd = @get_dsd(mem1d_dsd, .{ .tensor_access = |i|{1} -> result[i] });

const out_dsd = @get_dsd(fabout_dsd, if (@is_arch("wse3")) .{
    .extent = 1,
    .output_queue = d2h_1_oq
} else .{
    .fabric_color = @get_color(@bitcast(u16, sys_mod.MEMCPYD2H_1)),
    .extent = 1,
    .output_queue = d2h_1_oq
});

task main_task(data: f32) void {
    result[0] += data;
}

task send_result() void {
    @fmovs(out_dsd, result_dsd, .{ .async = true });
}

comptime {
    @bind_data_task(main_task, main_task_id);
    @bind_control_task(send_result, send_result_task_id);

    // On WSE-3, we must explicitly initialize input and output queues
    if (@is_arch("wse3")) {
        @initialize_queue(main_iq, .{ .color = main_color });
        @initialize_queue(d2h_1_oq, .{ .color = @get_color(@bitcast(u16, sys_mod.MEMCPYD2H_1)) });
    }
}

```

sentinel.csl

```

// WSE-2 task ID map
// On WSE-2, data tasks are bound to colors (IDs 0 through 24)
//

| <i>// ID var</i>             | <i>ID var</i> | <i>ID var</i>               | <i>ID var</i>               |
|------------------------------|---------------|-----------------------------|-----------------------------|
| <i>// 0</i>                  | <i>9</i>      | <i>18</i>                   | <i>27 reserved (memcpy)</i> |
| <i>// 1</i>                  | <i>10</i>     | <i>19</i>                   | <i>28 reserved (memcpy)</i> |
| <i>// 2 wtt_in_1_task_id</i> | <i>11</i>     | <i>20</i>                   | <i>29 reserved</i>          |
| <i>// 3 wtt_in_2_task_id</i> | <i>12</i>     | <i>21 reserved (memcpy)</i> | <i>30 reserved (memcpy)</i> |
| <i>// 4</i>                  | <i>13</i>     | <i>22 reserved (memcpy)</i> | <i>31 reserved</i>          |
| <i>// 5</i>                  | <i>14</i>     | <i>23 reserved (memcpy)</i> | <i>32</i>                   |
| <i>// 6</i>                  | <i>15</i>     | <i>24</i>                   | <i>33</i>                   |
| <i>// 7</i>                  | <i>16</i>     | <i>25</i>                   | <i>34</i>                   |
| <i>// 8</i>                  | <i>17</i>     | <i>26</i>                   | <i>35</i>                   |

  

// WSE-3 task ID map
// On WSE-3, data tasks are bound to input queues (IDs 0 through 7)

```

```

//
// ID var          ID var ID var          ID var
// 0 reserved (memcpy) 9      18          27 reserved (memcpy)
// 1 reserved (memcpy) 10     19          28 reserved (memcpy)
// 2 wtt_in_1_task_id 11     20          29 reserved
// 3 wtt_in_2_task_id 12     21 reserved (memcpy) 30 reserved (memcpy)
// 4                  13     22 reserved (memcpy) 31 reserved
// 5                  14     23 reserved (memcpy) 32
// 6                  15     24                  33
// 7                  16     25                  34
// 8                  17     26                  35

param memcpy_params;

const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);
const ctrl = @import_module("<control>");

// Sentinel to signal end of data
param sentinel: ul6;

// Colors
param main_color: color;

// Queue IDs
const h2d_1_iq: input_queue = @get_input_queue(2);
const h2d_2_iq: input_queue = @get_input_queue(3);
const main_oq: output_queue = @get_output_queue(4);

// Task IDs
// On WSE-2, data task IDs are created from colors; on WSE-3, from input queues
// Task ID for data task that recvs number of expected elements from host
const wtt_in_1_task_id: data_task_id =
    if (@is_arch("wse2")) @get_data_task_id(@get_color(@bitcast(ul6, sys_mod.MEMCPYH2D_1)))
    else if (@is_arch("wse3")) @get_data_task_id(h2d_1_iq);

// Task ID for data task that receives actual data from host
const wtt_in_2_task_id: data_task_id =
    if (@is_arch("wse2")) @get_data_task_id(@get_color(@bitcast(ul6, sys_mod.MEMCPYH2D_2)))
    else if (@is_arch("wse3")) @get_data_task_id(h2d_2_iq);

const sentinel_task_id: control_task_id = @get_control_task_id(sentinel);

var num_wvlts: il6 = 0;
var index: il6 = 0;

const fab_trans_wdsd = @get_dsd(fabout_dsd, if (@is_arch("wse3")) .{
    .extent = 1,
    .output_queue = main_oq
} else .{
    .extent = 1,
    .fabric_color = main_color,
    .output_queue = main_oq
});

const fab_trans_ctrl_wdsd = @get_dsd(fabout_dsd, if (@is_arch("wse3")) .{

```

```

        .extent = 1,
        .control = true,
        .output_queue = main_oq
    } else .{
        .extent = 1,
        .control = true,
        .fabric_color = main_color,
        .output_queue = main_oq
    });

// MEMCPYH2D_1 receives number of wavelets of MEMPCYH2D_2
task wtt_h2d_in_1(data: u32) void {
    num_wvlts = @as(i16, data);
}

// MEMCPYH2D_2 forwards data to main_color and appends a sentinel
// at the end.
task wtt_h2d_in_2(data: u32) void {
    @mov32(fab_trans_wdsd, data);
    index += 1;
    if (index >= num_wvlts) {
        // Construct ctrl_wlt with sentinel control task ID encoded
        const ctrl_wlt = ctrl.encode_control_task_payload(sentinel_task_id);
        @mov32(fab_trans_ctrl_wdsd, ctrl_wlt);
        index = 0;
    }
}

comptime {
    @bind_data_task(wtt_h2d_in_1, wtt_in_1_task_id);
    @bind_data_task(wtt_h2d_in_2, wtt_in_2_task_id);

    // On WSE-3, we must explicitly initialize input and output queues
    if (@is_arch("wse3")) {
        @initialize_queue(h2d_1_iq, .{ .color = @get_color(@bitcast(u16, sys_mod.MEMCPYH2D_1)) });
        @initialize_queue(h2d_2_iq, .{ .color = @get_color(@bitcast(u16, sys_mod.MEMCPYH2D_2)) });
        @initialize_queue(main_oq, .{ .color = main_color });
    }
}

```

run.py

```

#!/usr/bin/env cs_python

import argparse
import json
import numpy as np

from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime, MemcpyDataType # pylint: disable=no-name-in-module
from cerebras.sdk.runtime.sdkruntimepybind import MemcpyOrder # pylint: disable=no-name-in-module

```

```

parser = argparse.ArgumentParser()
parser.add_argument('--name', help='the test name')
parser.add_argument("--cmaddr", help="IP:port for CS system")
args = parser.parse_args()
dirname = args.name

# Parse the compile metadata
with open(f"{dirname}/out.json", encoding="utf-8") as json_file:
    compile_data = json.load(json_file)
    params = compile_data["params"]
    MEMCPYH2D_DATA_1 = int(params["MEMCPYH2D_DATA_1_ID"])
    MEMCPYH2D_DATA_2 = int(params["MEMCPYH2D_DATA_2_ID"])
    MEMCPYD2H_DATA_1 = int(params["MEMCPYD2H_DATA_1_ID"])
    size = int(params["size"])
    print(f"MEMCPYH2D_DATA_1 = {MEMCPYH2D_DATA_1}")
    print(f"MEMCPYH2D_DATA_2 = {MEMCPYH2D_DATA_2}")
    print(f"MEMCPYD2H_DATA_1 = {MEMCPYD2H_DATA_1}")
    print(f"size = {size}")

memcpy_dtype = MallocDataTypes.MEMCPY_32BIT
runner = SdkRuntime(dirname, cmaddr=args.cmaddr)

runner.load()
runner.run()

num_wvlts = 11
print(f"num_wvlts = number of wavelets for each PE = {num_wvlts}")

print("step 1: streaming H2D_1 sends number of input wavelets to P0")
h2d1_u32 = np.ones(size).astype(np.uint32) * num_wvlts
runner.memcpy_h2d(MEMCPYH2D_DATA_1, h2d1_u32.ravel(), 0, 0, 1, size, 1, \
    streaming=True, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=True)

# Use a deterministic seed so that CI results are predictable
np.random.seed(seed=7)

# Setup a {size}x11 input tensor that is reduced along the second dimension
input_tensor = np.random.rand(size, num_wvlts).astype(np.float32)
expected = np.sum(input_tensor, axis=1)

print("step 2: streaming H2D_2 to P0")
runner.memcpy_h2d(MEMCPYH2D_DATA_2, input_tensor.ravel(), 0, 0, 1, size, num_wvlts, \
    streaming=True, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=True)

print("step 3: streaming D2H at P1")
result_tensor = np.zeros(size, np.float32)
runner.memcpy_d2h(result_tensor, MEMCPYD2H_DATA_1, 1, 0, 1, size, 1, \
    streaming=True, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=False)

runner.stop()

# Ensure that the result matches our expectation
np.testing.assert_allclose(result_tensor, expected, atol=0.05, rtol=0)
print("SUCCESS!")

```

commands.sh

```
#!/usr/bin/env bash
```

```
set -e
```

```
cslc --arch=wse3 ./layout.csl --fabric-dims=11,12 \  
--fabric-offsets=4,1 -o out \  
--params=MEMCPYH2D_DATA_1_ID:2 \  
--params=MEMCPYH2D_DATA_2_ID:3 \  
--params=MEMCPYD2H_DATA_1_ID:4 \  
--params=size:4 \  
--memcpy --channels=1 --width-west-buf=0 --width-east-buf=0  
cs_python run.py --name out
```

© Copyright 2026, Cerebras Systems.

Last updated on Apr 15, 2026, 7:13:14 PM.